

# 深圳大学实验报告

课程名称：移动互联网应用

实验项目名称：课堂练习 1

学院：计算机与软件学院

专业：软件工程

指导教师：解为成

报告人：\_\_学号：\_\_班级：\_\_

实验时间：2026 年 3 月 30 日

实验报告提交时间：2026 年 3 月 30 日

教务部制

实验目的与要求:

1. 尝试使用 Intent 结合 Bundle 一起传递数据 (80 分);
2. 怎样模拟活动被回收, 使得 onRestart() 方法被替换成 onCreate 方法 (书本 P62-P63, 建议采用手机模拟器, 20 分)。

注意:

1. 实验报告中需要有实验结果的截屏图像;
2. 尽量删除空白页。

方法、步骤:

**1. 尝试使用 Intent 结合 Bundle 一起传递数据 (80 分):**

1) 创建并配置项目: 打开 Android Studio, 点击 New Project, 选择 Empty Views Activity 模板。将项目名称设为 Test1, 包名设置为 com.example.test1, 点击 Finish 等待 Gradle 构建完成并生成基础目录结构。

2) 编写界面布局: 打开 res/layout/activity\_main.xml, 编写主界面布局, 包含 3 个输入框 (分别用于输入姓名、年龄、备注内容) 和 1 个跳转按钮。右键项目包名, 选择 New -> Activity -> Empty Views Activity 创建 SecondActivity; 在自动生成的 activity\_second.xml 中编写接收界面布局, 添加 1 个 TextView 控件用于展示从前序页面接收到的组合数据。

**3) 实现逻辑代码:**

打开 MainActivity.java, 编写按钮点击逻辑: 通过 findViewById 获取输入框对象并提取用户输入的动态数据; 创建一个 Bundle 对象, 利用 putString() 和 putInt() 方法将数据以键值对形式存入; 随后创建 Intent 对象并通过 putExtras(bundle) 将数据包绑定, 调用 startActivity() 启动 SecondActivity。打开 SecondActivity.java 编写数据接收逻辑: 在 onCreate 方法中通过 getIntent().getExtras() 获取传递过来的 Bundle 对象, 根据对应的 Key 值解析出姓名、年龄和备注内容, 并将其格式化组合后展示到界面上的文本框内。

4) 运行与结果验证: 启动 Android 模拟器, 点击 Android Studio 顶部的 Run 按钮安装并运行应用。在主界面分别输入姓名、年龄及备注, 点击【发送全部数据并跳转】按钮。观察并确认 SecondActivity 能够准确、清晰地显示接收到的所有用户输入信息, 验证 Bundle 数据传递功能正常。

**2. 怎样模拟活动被回收, 使得 onRestart() 方法被替换成 onCreate 方法 (书本 P62-P63, 建议采用手机模拟器, 20 分):**

1) 重写状态保存方法: 打开 MainActivity.java, 重写 onSaveInstanceState(Bundle outState) 回调方法。在该方法中, 通过 findViewById 获取备注输入框中的文本内容, 并调用 outState.putString("data\_key", tempData) 将数据以键值对形式存入系统提供的 Bundle 对象中, 确保在活动被销毁前完成数据备份。

- 2) 编写状态恢复逻辑：在 MainActivity.java 的 onCreate(Bundle savedInstanceState) 方法中，添加恢复逻辑：通过判断 savedInstanceState 是否为空（null）来确认活动是否被意外回收；若不为空，则利用 getString("data\_key") 解析出之前保存的数据，并重新调用 setText() 方法将内容填充回输入框，实现界面的无缝恢复。
- 3) 配置模拟环境：启动 Android 模拟器，进入系统“设置 -> 关于手机”，连续点击“版本号” 7 次开启开发者选项。返回设置菜单进入“开发者选项”，找到“应用”栏目并开启“不保留活动”(Don't keep activities) 开关，以此强制模拟系统内存不足导致后台活动被立即销毁回收的真实场景。
- 4) 运行与测试验证：点击运行按钮安装应用，在主界面备注框输入“模拟测试数据”后点击跳转按钮进入第二页。随后按下模拟器的 Back（返回）键 回到主界面，通过观察 Logcat 日志确认 onCreate() 方法被重新执行（而非 onRestart），并检查主界面输入框内容是否依然存在，以此验证瞬时状态保存与恢复功能的正确性。

实验过程及内容：

**1. 尝试使用 Intent 结合 Bundle 一起传递数据（80 分）：**

**1) 编写主页面布局 res/layout/activity\_main.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="20dp"
    android:fitsSystemWindows="true"> <!-- 关键：避免被摄像头遮挡 -->

    <!--姓名输入 -->
    <EditText
        android:id="@+id/edit_name"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="请输入姓名"
        android:layout_marginTop="10dp" />

    <!--年龄输入 -->
```

```

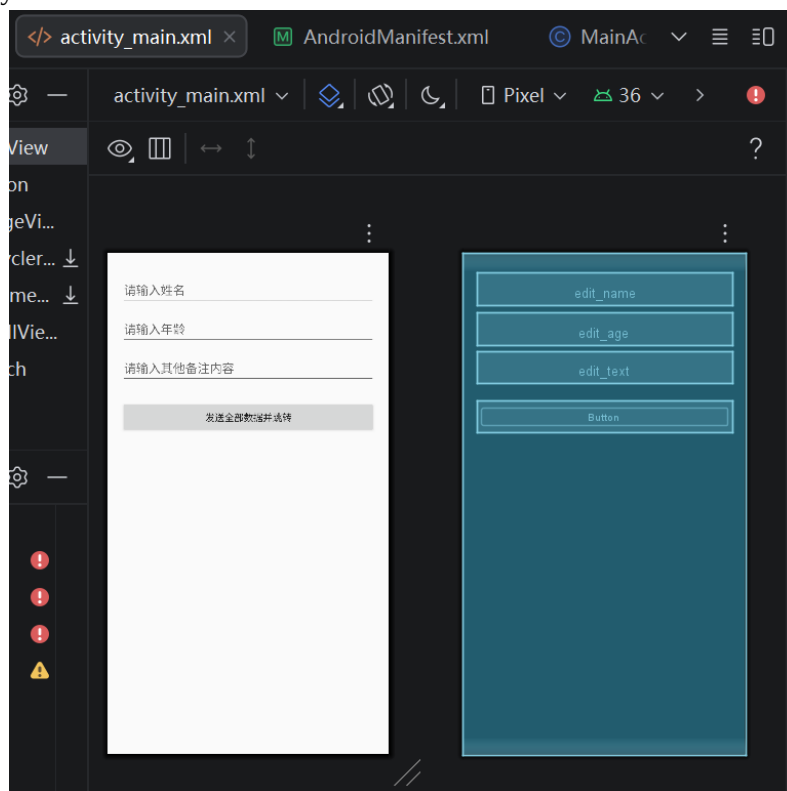
<EditText
    android:id="@+id/edit_age"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="请输入年龄"
    android:inputType="number"
    android:layout_marginTop="10dp" />

<!--备注/临时内容输入 -->
<EditText
    android:id="@+id/edit_text"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="请输入其他备注（测试回收恢复）"
    android:layout_marginTop="10dp" />

<Button
    android:id="@+id/btn_send_bundle"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="发送全部数据并跳转"
    android:layout_marginTop="24dp" />

```

</LinearLayout>



图一 主页面布局图

## 2) 编写主活动逻辑 MainActivity.java

```
package com.example.test1;

import androidx.appcompat.app.AppCompatActivity;
import android.content.Intent;
import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;

public class MainActivity extends AppCompatActivity {
    private EditText editName, editAge, editExtra;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        editName = findViewById(R.id.edit_name);
        editAge = findViewById(R.id.edit_age);
        editExtra = findViewById(R.id.edit_text);
        Button btnSend = findViewById(R.id.btn_send_bundle);

        // ---发送 Bundle 逻辑 ---
        btnSend.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                String name = editName.getText().toString();
                String ageStr = editAge.getText().toString();
                String extra = editExtra.getText().toString();

                Bundle bundle = new Bundle();
                bundle.putString("user_name", name);
                bundle.putInt("user_age", ageStr.isEmpty() ? 0 : Integer.parseInt(ageStr));
                bundle.putString("extra_info", extra);

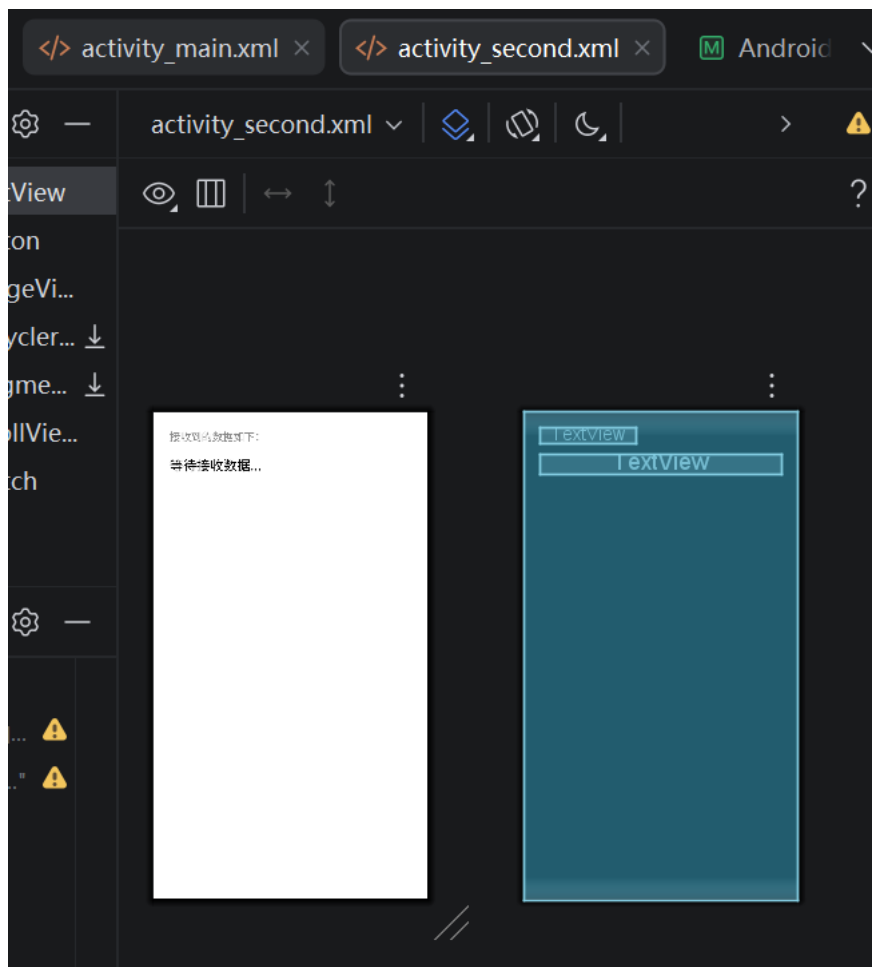
                Intent intent = new Intent(MainActivity.this, SecondActivity.class);
                intent.putExtras(bundle); // 绑定 Bundle
                startActivity(intent);
            }
        });
    }
}
```

### 3) 编写接收页面布局 res/layout/activity\_second.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="24dp"
    android:fitsSystemWindows="true"
    android:orientation="vertical">

    <TextView
        android:id="@+id/tv_display"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="18sp"
        android:lineSpacingExtra="8dp"
        android:text="等待接收数据..." />

</LinearLayout>
```



图二 接收页面布局图

#### 4) 编写接收活动逻辑 SecondActivity.java

```
package com.example.test1;

import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;
import android.widget.TextView;

public class SecondActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_second);

        TextView tvDisplay = findViewById(R.id.tv_display);

        // ---接收并解析数据 ---
        Bundle bundle = getIntent().getExtras();
        if (bundle != null) {
            String name = bundle.getString("user_name");
            int age = bundle.getInt("user_age");
            String info = bundle.getString("extra_info");

            String result = "接收到的用户信息: \n" +
                "姓名: " + name + "\n" +
                "年龄: " + age + "\n" +
                "备注: " + info;

            tvDisplay.setText(result);
        }
    }
}
```

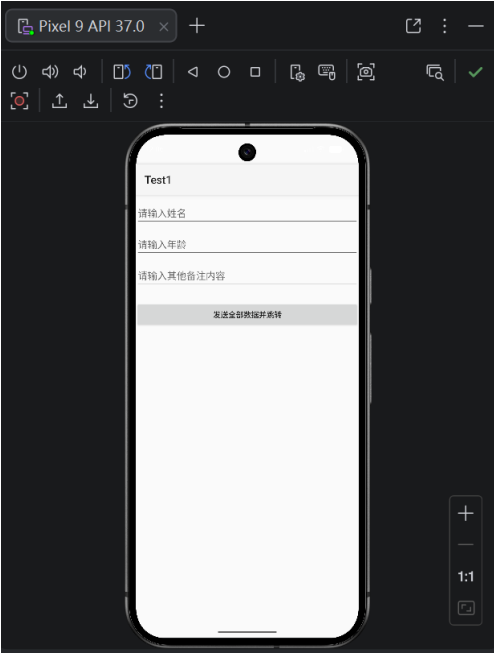
#### 5) 注册 Activity

在 <application> 标签内确保注册了两个 Activity:

```
<activity android:name=".MainActivity" android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>

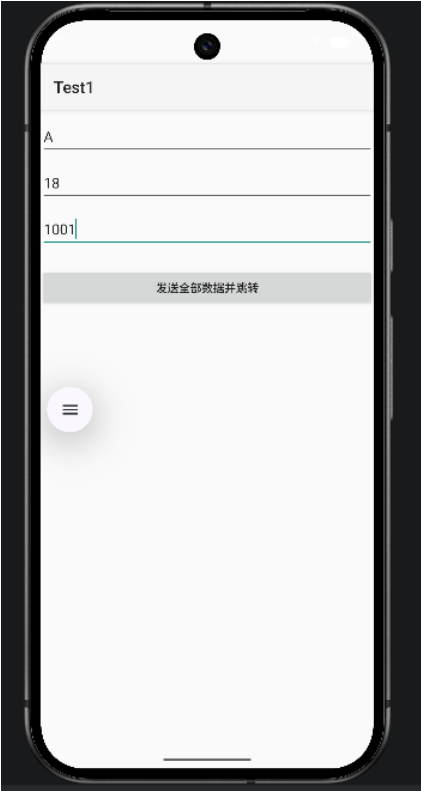
<!-- 注册 SecondActivity -->
<activity android:name=".SecondActivity" />
```

6) 启动应用并测试功能



图三 应用主页面

输入姓名、年龄和备注内容并点击发送按钮



图四 输入发送内容





图五 接收到内容

2. 怎样模拟活动被回收，使得 `onRestart()` 方法被替换成 `onCreate` 方法（书本 P62-P63，建议采用手机模拟器，20 分）：

1) 在主活动逻辑中添加保存和恢复逻辑代码

// ---保存数据逻辑 ---

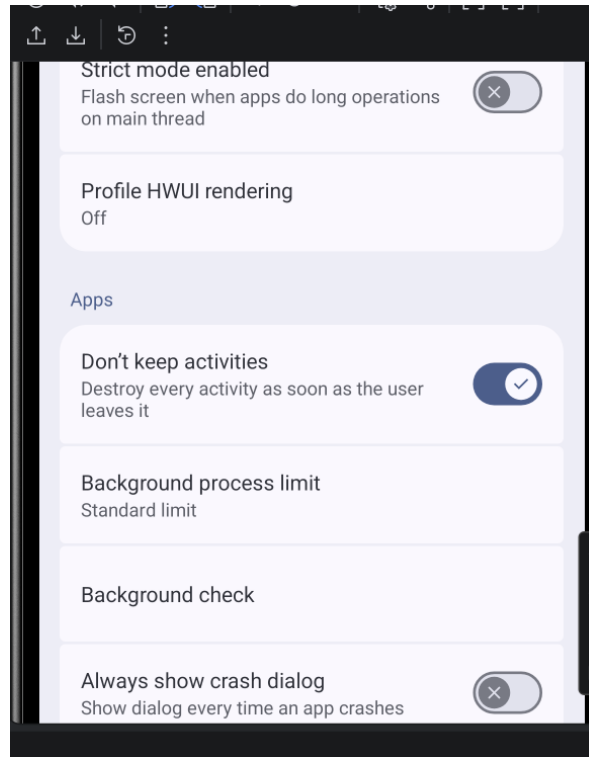
@Override

```
protected void onSaveInstanceState(Bundle outState) {  
    super.onSaveInstanceState(outState);  
    String tempData = editExtra.getText().toString();  
    outState.putString("data_key", tempData); // 保存备注内容到 Bundle  
}
```

// ---恢复数据逻辑 ---

```
if (savedInstanceState != null) {  
    String savedData = savedInstanceState.getString("data_key");  
    if (savedData != null) {  
        editExtra.setText(savedData); // 将保存的内容填回输入框  
    }  
}
```

2) 开启模拟器“开发者选项”中的“不保留活动”



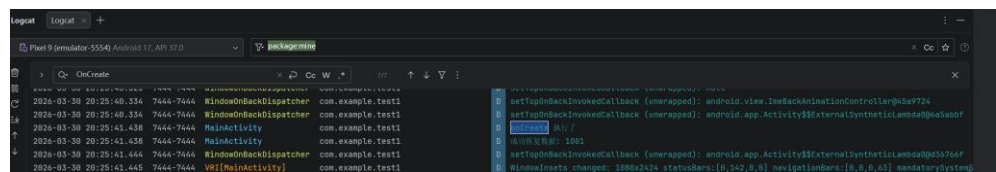
图六 开启“不保留活动”选项

3) 运行应用，输入并发送后跳转第二页，点击模拟器的“BACK”键回到上一页，可以看到输入的内容没有消失



图七 数据恢复

同时查看日志，可以发现 onCreate 的执行



图八 日志图

数据处理分析：

1.在 MainActivity 中，程序通过 findViewById 获取用户输入的动态数据。通过 Bundle 对象作为数据容器，利用 putString() 和 putInt() 方法实现了不同类型数据（字符串与整型）的封装。实验结果显示（图五），SecondActivity 成功解析了 key 为 "user\_name"、"user\_age" 和 "extra\_info" 的数据。这证明了 Intent 结合 Bundle 的传递机制具有良好的结构化特性，能够安全地在组件间传输复杂数据。

2.回收模拟通过开启开发者选项中的“不保留活动”，强制模拟了系统由于内存不足销毁后台活动的情况。在正常情况下，从 SecondActivity 返回主界面应触发 onRestart()。但在开启模拟环境后，观察日志图（图八）可以发现：系统并没有调用 onRestart()，而是重新调用了 onCreate()。这“实锤”证明了原有的 MainActivity 实例已被系统销毁，当前界面是一个全新的实例。由于重写了 onSaveInstanceState()，备注信息在销毁前被暂存在系统 Bundle 中。当新实例启动时，onCreate(Bundle savedInstanceState) 中的参数不再为 null。通过逻辑判断与 setText() 操作，原本应消失的文本被成功恢复（见图七）。数据分析表明，该机制是保障移动端应用在内存波动环境下用户体验一致性的核心。

### 实验结论：

1. Intent 结合 Bundle 是 Android 开发中页面跳转和数据传递的标准做法。Bundle 采用键值对（Key-Value）存储方式，逻辑清晰，且支持多种基本数据类型，能有效解耦数据采集与数据展示的逻辑。
2. Activity 的生命周期并非总是线性的。在系统内存压力或配置更改（如旋转屏幕、不保留活动）时，Activity 会被销毁并重建。理解并能模拟这种回收场景，是开发高质量应用的前提。
3. 实验验证了 onSaveInstanceState() 和 onCreate()/onRestoreInstanceState() 回调在处理瞬时数据保存中的关键作用。通过这些机制，可以有效解决应用因意外回收导致的输入内容丢失问题，极大提升了应用的容错能力和用户体验。
4. 通过本次实验，我深入掌握了 Android 活动（Activity）的生命周期及其数据交互机制。

1. Intent 结合 Bundle 是 Android 开发中页面跳转和数据传递的标准做法。Bundle 采用键值对（Key-Value）存储方式，逻辑清晰，且支持多种基本数据类型，能有效解耦数据采集与数据展示的逻辑。

2. **Activity** 的生命周期并非总是线性的。在系统内存压力或配置更改（如旋转屏幕、不保留活动）时，**Activity** 会被销毁并重建。理解并能模拟这种回收场景，是开发高质量应用的前提。

3.实验验证了 onSaveInstanceState() 和 onCreate()/onRestoreInstanceState() 回调在处理瞬时数据保存中的关键作用。通过这些机制，可以有效解决应用因意外回收导致的输入内容丢失问题，极大提升了应用的容错能力和用户体验。

4.通过本次实验，我深入掌握了 Android 活动（Activity）的生命周期及其数据交互机制

指导教师批阅意见：

成绩评定：

指导教师签字：解为成  
2026 年 4 月 日

成绩评定：

指导教师签字：解为成

2026 年 4 月 日

指导教师签字：解为成  
2026 年 4 月 日

|     |
|-----|
| 备注: |
|-----|

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。